

Lab Manual: Implementing the Banker's Algorithm (Safe State Detection)

1. Objective

Students will implement the **Banker's Algorithm** to determine whether a system of concurrent processes competing for multiple resource types is in a **safe state**. If it is safe, the program must also output a valid **safe sequence** in which the processes can complete without causing a deadlock.

2. Background

The Banker's Algorithm is a deadlock-avoidance algorithm used by an operating system to decide whether granting a resource request will leave the system in a state from which every process can still finish. A state is **safe** if there exists at least one order (a *safe sequence*) in which all processes can obtain the resources they still need, run to completion, and release their resources back to the system.

If no such order exists, the state is **unsafe** — it does not necessarily mean a deadlock has occurred, but it means one *could* occur.

3. Notation and Inputs

Let:

- **n** = number of processes (in this exercise, $n = 4$: P0, P1, P2, P3)
- **m** = number of resource types (e.g., R0, R1, R2 — 3 resource types in the example below, but your program should work for any m)

You will be given three inputs:

Symbol	Name	Size	Meaning
E	Existing Resources	$1 \times m$ vector	Total instances of each resource type installed in the system
P	Allocation / Possessed Matrix	$n \times m$ matrix	$P[i][j]$ = number of instances of resource type j currently allocated to process i
Need (or Max)	Need Matrix	$n \times m$ matrix	$Need[i][j]$ = number of additional instances of resource type j that process i may still request to complete its task

Symbol	Name	Size	Meaning
A	Available Resources	$1 \times m$ vector	$A[j]$ = instances of resource type j not currently allocated to any process

Important relationships your program should validate:

```
A[j] = E[j] - Σ(P[i][j]) for all i      (Available = Existing - sum
of each column of Allocation)
```

```
If a Max matrix is given instead of Need:
Need[i][j] = Max[i][j] - P[i][j]
```

If the input gives you **Max** instead of **Need** directly, compute **Need** first before running the safety algorithm.

4. The Safety Algorithm

The algorithm determines if the current state (given by E, P, Need, A) is safe.

5. Worked Example

Use this example to verify your implementation before testing it on other datasets.

System: 4 processes (P0–P3), 3 resource types (R0, R1, R2)

Existing resources (E):

R0	R1	R2
9	5	7

Allocation / Possessed matrix (P):

Process	R0	R1	R2
P0	0	1	0
P1	2	0	0
P2	3	0	2
P3	2	1	1

Maximum demand matrix (Max):

Process	R0	R1	R2
P0	7	5	3
P1	3	2	2
P2	9	0	2
P3	2	2	2

Need matrix (Max – Allocation), computed:

Process	R0	R1	R2
P0	7	4	3
P1	1	2	2
P2	6	0	0
P3	0	1	1

Available resources (A = E – column sums of P):

Column sums of P = (0+2+3+2, 1+0+0+1, 0+0+2+1) = (7, 2, 3)

$$A = E - \text{sum}(P) = (9-7, 5-2, 7-3) = (2, 3, 4)$$

Result: The system is **SAFE**, with safe sequence:

< P1, P3, P2, P0 >

Final Work vector equals E = (9, 5, 7), confirming all resources were eventually released. Use this exact result to sanity-check your code.

6. Implementation Guidelines

Students may implement this in any language (Python, C, C++, Java are all common choices). Regardless of language, structure your program as follows:

6.1 Input format (suggested)

Accept input either via hardcoded arrays, a text file, or standard input.

Then compute `Need` and `A` internally rather than requiring them as raw input — this forces students to implement the derivation formulas correctly, which is often where mistakes happen.

6.2 Output requirements

Your program should print:

1. The computed `Need` matrix
2. The computed `Available` vector
3. Whether the system is in a safe state (yes/no)
4. If safe, the safe sequence of process execution
5. If unsafe, a message stating no safe sequence exists